AFTERLIFE

Temporal Asset Protocol

V2 — Stellar Soroban Edition

PRODUCT REQUIREMENTS DOCUMENT

Prepared for Anti-Gravity Development · April 2026

> ⚠ **IMPORTANT TECHNICAL NOTE:** Stellar's smart contract platform (Soroban) is NOT EVM-based. Contracts are written in Rust and compiled to WebAssembly (Wasm) — not Solidity. This is a complete technology migration from the V1 EVM stack. The existing Solidity contracts cannot be directly ported; they must be rewritten in Rust using the soroban-sdk.

1. Executive Summary

AfterLife V2 is a ground-up rebuild of the AfterLife Temporal Asset Protocol on Stellar's Soroban smart contract platform. The original V1 deployed on Arbitrum Sepolia and Mantle Sepolia using EVM/Solidity. V2 migrates the entire stack to Stellar — using Rust/Wasm contracts, native Stellar wallets (Freighter, Albedo, xBull), and a dramatic new Three.js-powered UI.

The core product concept remains: a decentralised dead man's switch that distributes a user's crypto assets to chosen beneficiaries if they go inactive beyond a set threshold. V2 brings this to Stellar's ecosystem with a premium visual experience and multi-wallet support.

Attribute	Detail
Project	AfterLife V2 — Stellar Soroban Rebuild
Dev Tool	Anti-Gravity (using GitHub ZIP as source)
Chain	Stellar Mainnet / Testnet (Soroban)
Contract Language	Rust → WebAssembly via soroban-sdk
Frontend Stack	React + TypeScript + Vite + Three.js
Wallets	Freighter, Albedo, xBull, WalletConnect (Stellar)
Target Network	Stellar Mainnet + Testnet

2. Problem Statement & Goals

2.1 Problem

Billions in crypto assets become permanently inaccessible each year due to owner death or incapacitation — no inheritance mechanism exists natively in blockchain. AfterLife solves this with a trustless, time-gated dead man’s switch.

2.2 Why Migrate to Stellar?

- Lower transaction costs than Ethereum L2s
- Faster finality (target 2.5s with Protocol 23 upgrades)
- Strong institutional and stablecoin ecosystem (USDC native on Stellar)
- Soroban smart contracts have dedicated SDK, CLI tooling, and testnet sandbox

- Freighter wallet is actively maintained by the Stellar Development Foundation
- Growing DeFi ecosystem on Stellar in 2025–2026

2.3 V2 Goals

- Rebuild all smart contracts in Rust using soroban-sdk
- Multi-wallet support: Freighter (primary), Albedo, xBull
- Stunning Three.js-powered UI with particle systems and cosmic aesthetic
- Prominent AfterLife brand identity with new logo
- Full parity with V1 protocol logic: Owner / Guardian / Beneficiary roles
- Deploy on Stellar Testnet first, then Mainnet

3. Scope

3.1 In Scope

- Soroban smart contract — AfterLifeProtocol.rs
- Multi-wallet connector layer (Freighter, Albedo, xBull)
- Full React frontend with Three.js hero scene
- Dashboard: Owner, Guardian, Beneficiary views
- Logo design (SVG + PNG exports)
- Deployment scripts for Stellar CLI
- Testnet + Mainnet configuration

3.2 Out of Scope (V2)

- Cross-chain bridge from V1 EVM wallets
- Mobile app
- Backend indexer (Horizon API used directly)
- DAO governance

4. Technical Architecture

4.1 Technology Stack

Layer	Technology	Notes
Smart Contract	Rust + soroban-sdk	Compiled to Wasm
Contract Deploy	Stellar CLI	stellar contract deploy

Layer	Technology	Notes
Frontend Framework	React 18 + TypeScript	Vite bundler
3D / Visual	Three.js + @react-three/fiber	Particle cosmos scene
Stellar SDK	@stellar/stellar-sdk	Transaction building
Wallet — Primary	Freighter (@stellar/freighter-api)	SDF official browser extension
Wallet — Secondary	Albedo (albedo-link)	Web-based signer
Wallet — Tertiary	xBull Wallet SDK	Browser extension
Stellar RPC	Horizon API + Soroban RPC	State queries & tx submit
Animation	Framer Motion	UI transitions
Styling	CSS Modules + custom design tokens	Dark-first, cosmic theme
State Mgmt	Zustand + TanStack Query	Global + server state
Testing	Jest + soroban-sdk test harness	Unit + contract tests
Package Mgr	pnpm	Mirrors V1 tooling

4.2 Project Structure

The ZIP from the GitHub repository will serve as the starting reference. The V2 project structure will be:

```
afterlife-v2/ ├── contracts/                    ← Rust/Soroban smart contracts │   └──
afterlife_protocol/ │       ├── src/lib.rs           ← Main contract │       ├──
src/types.rs         ← Data structures │       ├── src/errors.rs │       └──
src/test.rs ├── src/ │   ├── components/ │   │   ├── three/               ← Three.js
scenes │   │   ├── wallet/              ← Freighter / Albedo / xBull │   │   ├──
dashboard/           ← Owner/Guardian/Beneficiary panels │   │   └── ui/
← Shared UI components │   ├── contexts/ │   │   ├── WalletContext.tsx    ← Multi-
wallet state │   │   └── ContractContext.tsx │   ├── hooks/ │   ├── services/ │   │
├── stellar.ts           ← Horizon/Soroban RPC calls │   │   └── contract.ts
← Contract interaction layer │   └── views/ ├── scripts/ │   └── deploy.sh
← Stellar CLI deploy script ├── public/ │   └── logo.svg └── Cargo.toml
```

5. Smart Contract Specification

5.1 Overview

The V2 contract is a single Soroban contract written in Rust. Unlike the EVM version (which used Solidity structs and mappings), Soroban stores state via its persistent ledger storage API. The contract must be compiled with the soroban-sdk and deployed using the Stellar CLI.

5.2 Data Structures

Key Rust types to define in src/types.rs:

```
#[derive(Clone, Debug)] pub struct VaultConfig {     pub owner: Address,     pub
inactivity_threshold: u64,   // in ledgers     pub guardians: Vec<Address>,     pub
beneficiaries: Vec<Beneficiary>,     pub state: ProtocolState,     pub last_activity:
u64,          // ledger number     pub locked_balance: i128,        // in stroops }
#[derive(Clone, Debug)] pub struct Beneficiary {     pub address: Address,     pub
allocation_bps: u32,         // basis points, sum ≤ 10000     pub has_claimed: bool, }
pub enum ProtocolState {     Active,     Warning,     Pending,     Executing,
Completed, }
```

5.3 Contract Functions

Function	Caller	Description
register(threshold)	Owner	Initialize vault with inactivity threshold (in ledgers)
deposit(amount)	Owner	Lock XLM into the vault
withdraw(amount)	Owner	Withdraw from vault (only in Active state)
add_guardian(addr)	Owner	Add a guardian address (max 10)
add_beneficiary(addr, bps)	Owner	Add beneficiary with allocation in basis points
prove_life()	Owner	Reset inactivity timer — keeps vault Active
confirm_inactivity(owner)	Guardian	Guardian votes that owner is inactive
claim(owner_addr)	Beneficiary	Claim allocated share after execution begins
revive()	Owner	Revert to Active within 7-day grace period
get_vault_state(owner)	Anyone	Read vault config and current state
get_claimable(owner, bene)	Anyone	Calculate claimable amount for a beneficiary

5.4 State Machine

The protocol state transitions mirror V1 exactly:

- ACTIVE → WARNING: inactivity reaches 70% of threshold
- WARNING → PENDING: guardian confirms inactivity (majority vote if multiple guardians)
- PENDING → EXECUTING: vesting period begins after confirmation
- EXECUTING → COMPLETED: all beneficiaries have claimed
- ANY → ACTIVE: owner calls prove_life() or revive() within grace period

IMPORTANT: Stellar uses ledger sequence numbers, not timestamps. All time-based logic must convert real-world durations to approximate ledger counts (Stellar closes ~1 ledger per 5 seconds, targeting 2.5s in 2026).

5.5 Platform Fee

A 10% platform fee is deducted from all beneficiary claims, sent to a protocol treasury address set at contract initialization. This mirrors the V1 fee model.

6. Multi-Wallet Integration

6.1 Freighter (Primary)

Freighter is the official Stellar Development Foundation browser extension wallet. It supports Soroban smart contract interactions natively.

- Package: @stellar/freighter-api
- Connect: isConnected(), requestAccess()
- Sign transactions: signTransaction(xdr, {network, networkPassphrase})
- Get public key: getPublicKey()
- Supports both Testnet and Mainnet with network switching

6.2 Albedo

Albedo is a web-based Stellar signer — no extension install required, making it ideal for onboarding non-technical users.

- Package: albedo-link
- Works entirely in-browser via popup/redirect flow
- Supports: publicKey(), signTransaction(), pay()
- Great for users who don't have Freighter installed

6.3 xBull Wallet

xBull is a community Stellar wallet browser extension with Soroban support.

- Package: @creit-tech/stellar-wallets-kit (recommended unified kit)
- The @creit-tech/stellar-wallets-kit package wraps Freighter, Albedo, xBull, and more under a single API — highly recommended to reduce integration complexity

6.4 Wallet Context Architecture

```
// WalletContext.tsx interface WalletState {   wallet: 'freighter' | 'albedo' |
'xbull' | null;   publicKey: string | null;   network: 'TESTNET' | 'MAINNET';
isConnected: boolean;   connect: (wallet: WalletType) => Promise<void>;   disconnect:
() => void;   signTransaction: (xdr: string) => Promise<string>; }
```

7. Frontend & UX Design

7.1 Aesthetic Direction

The V2 UI embraces a cosmic / afterlife theme: deep space backgrounds, particle nebula systems built in Three.js, gold accent colors symbolising timeless value, and dark glassmorphism card panels. The visual language should feel like a premium, otherworldly financial instrument.

7.2 Design Tokens

Token	Value	Usage
--color-bg-primary	#050508	Main background (near black)
--color-bg-surface	#0D0D14	Card / panel surfaces
--color-accent-gold	#C9A84C	Primary brand accent
--color-accent-glow	#FFD97A	Hover/active gold highlight
--color-text-primary	#F0EDE5	Main body text
--color-text-secondary	#888880	Muted labels
--color-stellar	#3DD9AC	Stellar brand teal
--color-danger	#E05A5A	Warning / danger states
--font-display	'Cinzel', serif	Headings, logo text
--font-body	'DM Sans', sans-serif	Body and UI text
--font-mono	'JetBrains Mono', mono	Addresses, hashes

7.3 Three.js Hero Scene

The landing page features a live Three.js scene as the full-screen background:

- 10,000+ star particle system with slow rotation
- Floating hourglass wireframe mesh (symbolising the death timer)
- Nebula dust cloud using additive blending particles
- Gold light point emanating from the hourglass centre
- Camera slowly dolly-zooms in on scroll
- Responsive canvas, degrades gracefully on low-end devices

7.4 Page / View Structure

- Landing / Hero: Three.js scene + tagline + wallet connect CTA
- Owner Dashboard: vault status, deposit/withdraw, add guardians & beneficiaries, prove life

- Guardian Panel: enter owner address, check inactivity status, confirm inactivity
- Beneficiary Panel: check allocation, claim assets
- Network Switcher: Testnet / Mainnet toggle in header
- Wallet Modal: Freighter / Albedo / xBull selection UI

8. Brand Identity & Logo

8.1 Logo Concept

The AfterLife logo combines three symbolic elements:

- An hourglass — representing the dead man's switch timer and passing of time
- A keyhole or open arch within the hourglass — representing the unlocking of assets
- A subtle cosmic ring / orbit — representing the blockchain and Stellar's orbital aesthetic

The logotype uses 'Cinzel' (a Roman serif with an ancient/eternal quality) in all caps. The wordmark reads: AFTERLIFE with a thin gold separator line beneath and 'TEMPORAL ASSET PROTOCOL' as a subtitle in tracking-heavy sans-serif.

8.2 Logo Colour Variants

- Primary: Gold (#C9A84C) icon on near-black (#050508) background
- Reversed: Dark icon on cream (#F0EDE5) for light contexts
- Mono: Single-colour black or white for embeds
- Stellar variant: Teal (#3DD9AC) + gold for co-branded Stellar materials

8.3 Logo Usage

- Header: full logo (icon + wordmark) at 40px height
- Favicon: icon only, optimised at 32x32 and 16x16
- OG Image: logo centred on cosmic background at 1200x630

9. Development Phases

Phase 1: Foundation (Days 1–3)

Working with Anti-Gravity, starting from the GitHub ZIP as a reference codebase:

- Set up Rust + soroban-sdk toolchain (rustup + wasm32 target + stellar CLI)
- Extract and review V1 contracts from ZIP for logic mapping
- Scaffold new Rust contract: AfterLifeProtocol
- Implement core data types and storage schema

- Write unit tests using soroban-sdk test harness

Phase 2: Contract Completion (Days 3–5)

- Implement all contract functions (see Section 5.3)
- State machine logic with ledger-based timing
- Platform fee logic
- Deploy to Stellar Testnet via CLI
- Manual testing via Stellar Lab

Phase 3: Frontend Scaffold (Days 5–7)

- Vite + React + TypeScript project setup (using V1 structure as reference)
- Three.js hero scene: particles + hourglass + nebula
- Wallet connector: Freighter first, then Albedo + xBull
- WalletContext + ContractContext setup
- Basic routing: Landing / Owner / Guardian / Beneficiary

Phase 4: Dashboard Build (Days 7–11)

- Owner Dashboard: all vault management actions
- Guardian Panel: inactivity monitoring and confirmation
- Beneficiary Panel: claim interface
- Connect all UI actions to Soroban contract via stellar-sdk
- Transaction building, signing, and submission flow

Phase 5: Polish & Deploy (Days 11–14)

- Final UI polish: animations, transitions, loading states
- Logo SVG integration across all contexts
- Error handling and user feedback
- Responsive design (desktop-first, mobile-compatible)
- Testnet end-to-end test with all three wallets
- Mainnet deployment
- Vercel / Netlify deploy

10. Anti-Gravity Development Workflow

10.1 Getting Started with the ZIP

Since you'll be using the GitHub repository ZIP as source material in Anti-Gravity:

- Download the ZIP from: https://github.com/Mrinmoy-programmer07/AfterLIfe/archive/refs/heads/main.zip
- Import the ZIP into Anti-Gravity as your starting context
- Use it as a reference for: contract logic, state machine, types, component structure, and existing hooks
- Do NOT carry over: hardhat.config.cjs, wagmi.config.ts, or any EVM-specific config

10.2 Key V1 Files to Migrate Logic From

V1 File	V2 Equivalent / Action
contracts/AfterLifeProtocol.sol	Rewrite as contracts/afterlife_protocol/src/lib.rs
types.ts	Port type definitions to Rust types.rs + TS types for frontend
contexts/	Rebuild WalletContext for Stellar wallets
hooks/	Rebuild hooks to use stellar-sdk instead of wagmi
services/	Replace EVM services with Horizon/Soroban RPC calls
views/	Keep view structure, rebuild wallet connection logic
hardhat.config.cjs	DISCARD — replaced by Stellar CLI deploy scripts
wagmi.config.ts	DISCARD — replaced by @stellar/stellar-sdk

10.3 Critical Stellar/Soroban Differences from EVM

Concept	EVM (V1)	Stellar Soroban (V2)
Contract Language	Solidity	Rust
Compilation	EVM bytecode	WebAssembly (Wasm)
Time Unit	block.timestamp (seconds)	Ledger sequence numbers
Storage	Mappings / state vars	Persistent/temporary ledger entries
Native Token	ETH / MNT	XLM (stroops = 1/10,000,000 XLM)
Address Format	0x... (hex, 20 bytes)	G... (Stellar public key, 56 chars)
Wallet Library	wagmi / viem	@stellar/freighter-api + albedo-link
Tx Building	ethers.js / viem	@stellar/stellar-sdk (XDR format)
Deploy Tool	Hardhat / Foundry	stellar contract deploy (CLI)
Events	Solidity events	Soroban contract events (diagnostic)

11. Risks & Mitigations

Risk	Severity	Mitigation
Rust/Soroban learning curve	High	Use official Stellar docs + example contracts repo; start with simple contract first
Ledger timing inaccuracy	Medium	Document the ledger-to-time conversion clearly; allow owner to set threshold in ledger counts directly
Wallet API breaking changes	Medium	Pin package versions; use @creit-tech/stellar-wallets-kit as abstraction layer
Testnet contract archival	Low	Redeploy script ready; state archival is a known Stellar testnet behaviour
Three.js performance on mobile	Low	Detect device capability; serve static fallback hero image on low-end devices
XLM price volatility affecting fees	Low	Fee is % of assets, not fixed XLM — inherently self-adjusting

12. Appendix — Key Resources

Stellar / Soroban

- Soroban docs: https://developers.stellar.org/docs/build/smart-contracts
- soroban-sdk crate: https://crates.io/crates/soroban-sdk
- Stellar CLI: https://github.com/stellar/stellar-cli
- Example contracts: https://github.com/stellar/soroban-examples
- Stellar Lab (testnet tool): https://lab.stellar.org

Wallet SDKs

- Freighter API: https://github.com/stellar/freighter
- Albedo: https://github.com/orbitlens/albedo
- Stellar Wallets Kit (unified): https://github.com/Creit-Tech/Stellar-Wallets-Kit

Frontend

- @react-three/fiber: https://docs.pmnd.rs/react-three-fiber
- Three.js docs: https://threejs.org/docs
- Cinzel font (Google Fonts): https://fonts.google.com/specimen/Cinzel
- DM Sans font: https://fonts.google.com/specimen/DM+Sans

V1 Reference

- AfterLife V1 GitHub: https://github.com/Mrinmoy-programmer07/AfterLIfe
- V1 Live Demo: https://after-life-delta.vercel.app